



Chapter 7. Measuring Coverage with Coverage.py

How confident in a code change are you when your tests pass?

If you look at tests as a way to detect bugs, you can describe their sensitivity and specificity.

The *sensitivity* of your test suite is the probability of a test failure when there's a defect in the code. If large parts of the code are untested, or if the tests don't check for expected behavior, you have low sensitivity.

The *specificity* of your tests is the probability that they will pass if the code is free of defects. If your tests are *flaky* (they fail intermittently) or *brittle* (they fail when you change implementation details), then you have low specificity. Invariably, people stop paying attention to failing tests. This chapter isn't about specificity, though.

There's a great way to boost the sensitivity of your tests: when you add or change behavior, write a failing test before the code that makes it pass. If you do this, your test suite will capture your expectations for the code.

Another effective strategy is to test your software with the various inputs and environmental constraints that you expect it to encounter in the real world. Cover the edge cases of a function, like empty lists or negative numbers. Test common error scenarios, not just the "happy path."

Code coverage is a measure of the extent to which the test suite exercises your code. Full coverage doesn't guarantee high sensitivity: if your tests cover every line in your code, you can still have bugs. It's an upper bound, though. If your code coverage is 80%, then 20% of your code will

never trigger a test failure, no matter how many bugs creep in. It's also a quantitative measure amenable to automated tools. These two properties make coverage a useful proxy for sensitivity.

In short, coverage tools record each line in your code when you run it. After completion, they report the overall percentage of executed lines with respect to the entire codebase.

Coverage tools aren't limited to measuring test coverage. For example, code coverage lets you find which modules an API endpoint in a large codebase uses. Or you could use it to determine the extent to which code examples document your project.

In this chapter, I'll explain how to measure code coverage with Coverage.py, a coverage tool for Python. In the main sections of this chapter, you'll learn how to install, configure, and run Coverage.py, as well as how to identify missing lines of source code and missing branches in the control flow. I'll explain how to measure code coverage across multiple environments and processes. Finally, I'll talk about what code coverage you should aim for and how to reach your coverage target.

How does coverage measurement work in Python? The interpreter lets you register a callback—a *trace function*—using the function `sys.settrace`. From that point onward, the interpreter invokes the callback whenever it executes a line of code—as well as in some other situations, like entering or returning from functions or raising exceptions. Coverage tools register a trace function that records each executed line of source code in a local database.

Python's standard library includes a coverage tool in the `trace` module. Let's use it to measure test coverage for the `random-wikipedia-article` project. Run the test suite via `trace` in the active environment, as shown here:

```
$ py -m trace --count --summary --missing -C coverage --module pytest
===== test session starts =====
platform darwin -- Python 3.12.2, pytest-8.1.1, pluggy-1.4.0
rootdir: ...
configfile: pyproject.toml
plugins: Faker-24.2.0, anyio-4.3.0, xdist-3.5.0, pytest_httpserver-1.0.10
collected 21 items

tests/test_main.py ..... [100%]

===== 21 passed in 3.75s =====
lines   cov%   module    (path)
    26    92%   random_wikipedia_article.__init__  (...)
    33   100%   tests.test_main    (...)
...
```

The command asks `trace` to count how often each line is executed. It writes the results to `<module>.cover` files in the `coverage` directory, marking missed lines with the string `>>>>>`. It also writes a summary to the terminal, with coverage percentages for every module.

The summary includes modules from the standard library and third-party packages. It's easy to miss the fact that your `__main__` module doesn't appear at all. If you're curious why the `__init__` module only has 92% coverage, take a look at the file `random_wikipedia_article.__init__.cover`. (Bear with me, we'll get to those missing lines shortly.)

Measuring coverage with the standard library alone is cumbersome, even for a simple script like `random-wikipedia-article`. The `trace` module is an early proof of concept for Python code coverage. For real-world projects, you should use the third-party package `Coverage.py`.

Using Coverage.py

[Coverage.py](#) is a mature and widely used code coverage tool for Python. Created over two decades ago—predating PyPI and setuptools—and actively maintained ever since, it has measured coverage on every interpreter since Python 2.1.

Add `coverage[toml]` to your test dependencies (see [“Managing Test Dependencies”](#)). The `toml` extra allows Coverage.py to read its configuration from *pyproject.toml* on older interpreters. Since Python 3.11, the standard library includes the `tomllib` module for parsing TOML files.

Measuring coverage is a two-step process. First, you gather coverage data during a test run with `coverage run`. Second, you compile an aggregated report from the data with `coverage report`. Each command has a table in *pyproject.toml* under `tool.coverage`.

Start by configuring which packages you want to measure—it lets Coverage.py report modules that never showed up during execution, like the `__main__` module earlier. (Even without the setting, it won’t drown you in reports about the standard library.) Specify your top-level import package as well as the test suite:

```
[tool.coverage.run]
source = ["random_wikipedia_article", "tests"]
```

TIP

Measuring code coverage for your test suite may seem strange—but you should always do it. It alerts you when tests don’t run and helps you identify unreachable code within them. Treat your tests the same way you would treat any other code.¹

You can invoke `coverage run` with a Python script, followed by its command-line arguments. Alternatively, you can use its `-m` option with an importable module. Use the second method—it ensures that you run pytest from the current environment:

```
$ py -m coverage run -m pytest
```

After running this command, you'll find a file named `.coverage` in the current directory. Coverage.py uses it to store the coverage data it gathered during the test run.²

Coverage reports display the overall percentage of code coverage, as well as a breakdown per source file. Use the `show_missing` setting to also include line numbers for statements missing from coverage:

```
[tool.coverage.report]
show_missing = true
```

Run `coverage report` to show the coverage report in the terminal:

```
$ py -m coverage report
```

Name	Stmts	Miss	Cover	Missing
src/random_wikipedia_article/__init__.py	26	2	92%	37-38
src/random_wikipedia_article/__main__.py	2	2	0%	1-3
tests/__init__.py	0	0	100%	
tests/test_main.py	33	0	100%	
TOTAL	61	4	93%	

Overall, your project has a coverage of 93%—four statements never showed up during the tests. The test suite itself has full coverage, as you would expect.

Let's take a closer look at those missing statements. The `Missing` column in the coverage report lists them by line number. You can use your code editor to display the source code with line numbers, or the standard `cat -n` command on Linux and macOS. Again, the entire `__main__` module is missing from coverage:

```
1 from random_wikipedia_article import main # missing
2
```

```
3 main()
```

```
# missing
```

The missing lines in `__init__.py` correspond to the body of the `main` function:

```
36 def main():
37     article = fetch(API_URL) # missing
38     show(article, sys.stdout) # missing
```

This is surprising—the end-to-end test from [Example 6-2](#) runs the entire program, so all of those lines are definitely being tested. For now, disable coverage measurements for the `__main__` module:

```
[tool.coverage.run]
omit = ["*/__main__.py"]
```

You can exclude the `main` function using a special comment:

```
def main(): # pragma: no cover
    article = fetch(API_URL)
    show(article, sys.stdout)
```

If this feels like cheating, bear with me until [“Measuring in Subprocesses”](#), where you’ll re-enable coverage measurements for these lines.

If you run both steps again, Coverage.py will report full code coverage. Let’s make sure you’ll notice any lines that aren’t exercised by your tests. Configure Coverage.py to fail if the percentage drops below 100% again:

```
[tool.coverage.report]
fail_under = 100
```

Branch Coverage

If an article has an empty summary, `random-wikipedia-article` prints a trailing blank line (yikes). Those empty summaries are rare, but they exist, and this should be a quick fix. [Example 7-1](#) modifies `show` to print only nonempty summaries.

Example 7-1. Printing only nonempty summaries in `show`

```
def show(article, file):
    console = Console(file=file, width=72, highlight=False)
    console.print(article.title, style="bold", end="\n\n")
    if article.summary:
        console.print(article.summary)
```

Curiously, the coverage stays at 100%—even though you didn’t write a test first.

By default, Coverage.py measures *statement coverage*—the percentage of statements in your modules that the interpreter executed during the tests. If the summary isn’t empty, every statement in the function gets executed.

On the other hand, the tests exercised only one of two code paths through the function—they never skipped the `if` body. Coverage.py also supports *branch coverage*, which looks at all the transitions between statements in your code and measures the percentage of those traversed during the tests. You should always enable it, as it’s more precise than statement coverage:

```
[tool.coverage.run]
branch = true
```

Re-run the tests, and you’ll see Coverage.py flag the missing transition from the `if` statement on line 33 to the exit of the function:

```
$ py -m coverage run -m pytest
$ py -m coverage report
```

Name	Stmts	Miss	Branch	BrPart	Cover	Missing
src/.../__init__.py	24	0	6	1	97%	33->exit
tests/__init__.py	0	0	0	0	100%	
tests/test_main.py	33	0	6	0	100%	

TOTAL	57	0	12	1	99%	
Coverage failure: total of 99 is less than fail-under=100						

[Example 7-2](#) brings coverage back to 100%. It includes an article with an empty summary and adds the missing test for trailing blank lines.

Example 7-2. Testing articles with empty summaries

```

article = parametrized_fixture(
    Article("test"), *ArticleFactory.build_batch(10)
)

def test_trailing_blank_lines(article, file):
    show(article, file)
    assert not file.getvalue().endswith("\n\n")

```

Run the tests again—and they fail! Can you spot the bug in [Example 7-1](#)?

Empty summaries produce two blank lines: one to separate the title and the summary, and one from printing the empty summary. You’ve removed only the second one. [Example 7-3](#) removes the first one as well. Thanks, Coverage.py!

Example 7-3. Avoiding both trailing blank lines in `show`

```

def show(article, file):
    console = Console(file=file, width=72, highlight=False)
    console.print(article.title, style="bold")
    if article.summary:
        console.print(f"\n{article.summary}")

```


Testing in Multiple Environments

You will often need to support a variety of Python versions. Python releases come out every year, while long term support (LTS) distros can reach back a decade into Python’s history. End-of-life Python versions can have a surprising afterlife—distributors may provide security patches years after the core Python team ends support.

Let’s update `random-wikipedia-article` to support Python 3.7, which reached its end-of-life in June 2023. I’m assuming your project requires Python 3.10, with lower bounds on all dependencies. First, relax the Python requirement in *pyproject.toml*:

```
[project]
requires-python = ">=3.7"
```

Next, check if your dependencies are compatible with the Python version. Use `uv` to compile a separate requirements file for a Python 3.7 environment:

```
$ uv venv -p 3.7
$ uv pip compile --extra=tests pyproject.toml -o py37-dev-requirements.txt
× No solution found when resolving dependencies: ...
```

The error indicates that your preferred version of HTTPX has already dropped Python 3.7. Remove your lower version bound and try again. After a few similar errors and removing the lower bounds of other packages, dependency resolution finally succeeds. Restore the lower bounds using the older versions of these packages.

You’ll also need the backport `importlib-metadata` (see [“Environment Markers”](#)). Add the following entry to the `project.dependencies` field:

```
importlib-metadata>=6.7.0; python_version < '3.8'
```

Update the `__init__.py` module to fall back to the backport:

```
if sys.version_info >= (3, 8):  
    from importlib.metadata import metadata  
else:  
    from importlib_metadata import metadata
```

Compile the requirements one more time. Finally, update your project environment:

```
$ uv pip sync py37-dev-requirements.txt  
$ uv pip install -e . --no-deps
```

Parallel Coverage

If you now re-run `Coverage.py` under Python 3.7, it reports the first branch of the `if` statement as missing. This makes sense: your code executes the `else` branch and imports the backport instead of the standard library.

It may be tempting to exclude this line from coverage measurements—but don't. Third-party dependencies like the backport can break your code, too. Instead, collect coverage data from both environments.

First, switch the environment back to Python 3.12 using your original requirements:

```
$ uv venv -p 3.12  
$ uv pip sync dev-requirements.txt  
$ uv pip install -e . --no-deps
```

By default, `coverage run` overwrites any existing coverage data—but you can tell it to append the data instead. Re-run `Coverage.py` with the `--append` option and confirm you have full test coverage:

```
$ py -m coverage run --append -m pytest  
$ py -m coverage report
```

With a single file for coverage data, it's easy to erase data accidentally. If you forget to pass the `--append` option, you'll have to run the tests again. You could configure Coverage.py to append by default, but that's error-prone, too: if you forget to run `coverage erase` periodically, you'll end up with stale data in your report.

There's a better way to gather coverage across multiple environments. Coverage.py lets you record coverage data in separate files on each run. Enable this behavior with the `parallel` setting:³

```
[tool.coverage.run]
parallel = true
```

Coverage reports are always based on a single data file, even in parallel mode. You merge the data files using the command `coverage combine`. That turns the two-step process from earlier into a three-step one: first, `coverage run`; second, `coverage combine`; third, `coverage report`.

Let's put all of this together. For each Python version, set up the environment and run the tests, as shown here for Python 3.7:

```
$ uv venv -p 3.7
$ uv pip sync py37-dev-requirements.txt
$ uv pip install -e . --no-deps
$ py -m coverage run -m pytest
```

At this point, you'll have multiple `.coverage.*` files in your project. Aggregate them into a single `.coverage` file using the command `coverage combine`:

```
$ py -m coverage combine
Combined data file .coverage.somehost.26719.001909
Combined data file .coverage.somehost.26766.146311
```

Finally, produce the coverage report with `coverage report`:

```
$ py -m coverage report
```

Does it sound *incredibly* tedious to gather coverage like this? In [Chapter 8](#), you'll learn how to automate testing across multiple Python environments. You'll run this entire process using a single three-letter command.

Measuring in Subprocesses

At the end of [“Using Coverage.py”](#), you had to disable coverage for the `main` function and the `__main__` module. But the end-to-end test certainly exercises this code. Let's remove the `# pragma` comment and the `omit` setting and figure this out.

Think about how Coverage.py registers a trace function that records executed lines. Maybe you can already guess what's going on here: the end-to-end test runs your program in a separate process. Coverage.py never registered its trace function on the interpreter in that process. None of those executed lines were recorded anywhere.

Coverage.py provides a public API to enable tracing in the current process: the `coverage.process_startup` function. You could call the function when your application starts up. But there must be a better way—you shouldn't have to modify your code to support code coverage.

It turns out you don't need to. You can place a `.pth` file in the environment that calls the function during interpreter startup. This leverages a little-known Python feature (see [“Site Packages”](#)): the interpreter executes lines starting with an `import` statement in a `.pth` file.

Install a `_coverage.pth` file into the `site-packages` directory of your environment, with the following contents:

```
import coverage; coverage.process_startup()
```

You can find the `site-packages` directory under `lib/python3.x` on Linux and macOS, and under `Lib` on Windows.

Additionally, you need to set the environment variable `COVERAGE_PROCESS_START`. On Linux and macOS, use this syntax:

```
$ export COVERAGE_PROCESS_START=pyproject.toml
```

On Windows, use the following syntax instead:

```
> $env:COVERAGE_PROCESS_START = 'pyproject.toml'
```

Re-run the test suite, combine the data files, and display the coverage report. Thanks to measuring coverage in the subprocess, the program should have full coverage again.

NOTE

Measuring coverage in subprocesses works only in parallel mode. Without parallel mode, the main process overwrites the coverage data from the subprocess, because both use the same data file.

THE PYTEST-COV PLUGIN

The pytest plugin `pytest-cov` enables coverage measurements with Coverage.py when you run pytest. Add the plugin to your test dependencies and run pytest with the `--cov` option to enable it. You can still configure Coverage.py in *pyproject.toml*. Alternatively, the plugin exposes `--cov-*` options for various configuration settings.

The plugin aims to make everything work out of the box, behind the scenes—including subprocess coverage. The convenience comes at the price of a layer of indirection. Running `coverage` directly provides finer-grained control.

What Coverage to Aim For

Any coverage percentage below 100% means your tests won't detect bugs in some parts of your codebase. If you're working on a new project, there isn't any other meaningful coverage target.

That doesn't imply you should test every single line of code. Consider a log statement for debugging a rare situation. The statement may be difficult to exercise from a test. At the same time, it's probably low-risk, trivial code. Writing that test won't increase your confidence in the code significantly. Exclude the line from coverage using a *pragma* comment:

```
if rare_condition:
    print("got rare condition") # pragma: no cover
```

Don't exclude code from coverage just because it's cumbersome to test. When you start working with a new library or interfacing with a new system, it usually takes some time to figure out how to test your code. But often those tests end up detecting bugs that would have gone unnoticed and caused problems in production.

Legacy projects often consist of a large codebase with minimal test coverage. As a general rule, coverage in such projects should increase *monotonically*—no individual change should lead to a drop in coverage.

You'll often find yourself in a dilemma here: to test, you need to refactor the code, but refactoring is too risky without tests. Find the minimal safe refactoring to increase testability. Often, this consists of breaking a dependency of the code under test.⁴

For example, you may be testing a large function that, among other things, also connects to the production database. Add an optional parameter that lets you pass the connection from the outside. Tests can then pass a connection to an in-memory database instead.

[Example 7-4](#) recaps the Coverage.py settings you've used in this chapter.

Example 7-4. Configuring Coverage.py in *pyproject.toml*

```
[tool.coverage.run]
source = ["random_wikipedia_article", "tests"]
branch = true
parallel = true
omit = ["*/__main__.py"] # avoid this if you can

[tool.coverage.report]
show_missing = true
fail_under = 100
```

Summary

You can measure the extent to which the test suite exercises your project using Coverage.py. Coverage reports are useful for discovering untested lines. Branch coverage captures the control flow of your program instead of isolated lines of source code. Parallel coverage lets you measure coverage across multiple environments. You need to combine the data files before reporting. Measuring coverage in subprocesses requires setting up a *.pth* file and an environment variable.

Measuring test coverage effectively for a project requires some amount of configuration, as well as the right tool incantations. In the next chapter, you'll see how you can automate these steps with Nox. You'll set up checks that give you confidence in your changes—while staying out of your way.

- ¹ Ned Batchelder, [“You Should Include Your Tests in Coverage”](#), August 11, 2020.
- ² Under the hood, the *.coverage* file is just a SQLite database. Feel free to poke around if you have the `sqlite3` command-line utility ready on your system.
- ³ The name `parallel` is somewhat misleading; the setting has nothing to do with parallel execution.
- ⁴ Martin Fowler, [“Legacy Seam”](#), January 4, 2024.

